Journal of the Operational Research Society

ISSN: 0160-5682 (Print) 1476-9360 (Online) Journal homepage: https://www.tandfonline.com/loi/tjor20

A tabu search algorithm for the single vehicle routing allocation problem

L Vogt, C A Poojari & J E Beasley

To cite this article: L Vogt, C A Poojari & J E Beasley (2007) A tabu search algorithm for the single vehicle routing allocation problem, Journal of the Operational Research Society, 58:4, 467-480, DOI: 10.1057/palgrave.jors.2602165

To link to this article: https://doi.org/10.1057/palgrave.jors.2602165

Journal of the Operational Research Society (2007) 58, 467–480

www.palgrave-journals.com/jors

A tabu search algorithm for the single vehicle routing allocation problem

L Vogt[1], CA Poojari[2] and JE Beasley[2]*

[1]*EPFL-SB-IMA, CH-1015 Lausanne, Switzerland; and* [2]*Brunel University, Uxbridge, UK*

In the single vehicle routing allocation problem (SVRAP) we have a single vehicle, together with a set of customers, and the problem is one of deciding a route for the vehicle (starting and ending at given locations) such that it visits some of the customers. Customers not visited by the vehicle can either be allocated to a customer on the vehicle route, or they can be isolated. The objective is to minimize a weighted sum of routing, allocation and isolation costs. One special case of the general SVRAP is the median cycle problem, also known as the ring star problem, where no isolated vertices are allowed. Other special cases include the covering tour problem, the covering salesman problem and the shortest covering path problem. In this paper, we present a tabu search algorithm for the SVRAP. Our tabu search algorithm includes aspiration, path relinking and frequency-based diversification. Computational results are presented for test problems used previously in the literature and our algorithm is compared with the results obtained by other researchers. We also report results for much larger problems than have been considered by others.
Journal of the Operational Research Society (2007) **58,** 467–480. doi:10.1057/palgrave.jors.2602165
Published online 22 February 2006

Keywords: tabu search; median cycle problem; ring star problem; covering tour problem; covering salesman problem; shortest covering path problem

Introduction

In this paper, the single vehicle routing allocation problem (SVRAP) is presented and a tabu search algorithm presented for its solution. SVRAP is the problem of deciding a route for a single vehicle (starting at a given location and ending at a given location) such that it visits a set of customers. However, in contrast to the usual vehicle routing problem, not all of the customers need to be visited. Customers not visited by the vehicle can either be allocated to a customer on the vehicle route, or they can be isolated. The objective is to minimize a weighted sum of routing, allocation and isolation costs.

Figure 1 shows an example SVRAP solution. In that figure, the vehicle travels from the start node to the end node. There are three customers on the vehicle route (on-tour customers), and seven customers off-tour. Three off-tour customers are allocated to customers that are on-tour and four off-tour customers are isolated. Note here that Figure 1, for generality, has different start and end nodes. In practice we might well have the same start and end node.

SVRAP has a number of practical applications. For example, the routing of essential medical care services (eg a mobile clinic) to service the population in a rural area. In applications of this kind economic considerations are such that the vehicle cannot visit all locations (customers) and so people in unvisited locations must choose either to travel to their nearest visited location for treatment, or to remain untreated. Another example, but one where isolated customers might not be allowed, would be the design of postal collection routes such that all customers lie within a reasonable distance of their nearest collection point. If we force all customers to be visited (ie all customers have to be on-tour) then SVRAP reduces to the well-known (open) travelling salesman problem (TSP).

Note here that we have chosen to consider the problem as a *routing allocation* problem, that is, to decide a route and an appropriate allocation for each customer—either on-tour, or off-tour (allocated or isolated). We are aware that deciding such an allocation may well also involve deciding an *underlying location* for some facility (eg the location of collection points in the postal example referred to above). For this reason we could (potentially) label the problem using some combination of the words *routing, location and allocation*, for example see Laporte (1988). However, since the key decisions relate to routing and allocation, and since a number of applications do not involve deciding underlying locations, we have chosen to refer to the problem as a vehicle routing allocation problem.

Formulation and literature survey

In this section, SVRAP is formulated as a zero–one linear integer program. We also survey the literature relating to SVRAP and discuss related problems.

*Correspondence: JE Beasley, Department of Mathematical Sciences, Brunel University, Uxbridge UB8 3PH, UK.
E-mail: john.beasley@brunel.ac.uk

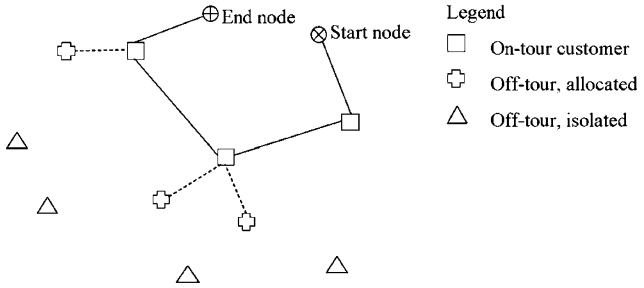

Figure 1 SVRAP example.

Formulation

We view SVRAP as a (directed) routing problem on a graph. The customers and the starting and ending point of the (single vehicle) tour are modelled as vertices on a graph with vertex set $V=\{0, 1, \ldots, n, n+1\}$. The customers are vertices 1–n, the starting point of the tour is vertex 0 and the ending point of the tour is vertex $n+1$. Define:

- c_{ij} the cost of tour arc (i,j);
- d_{ij} the cost of allocating (off-tour) vertex j to (on-tour) vertex i;
- D_i the cost of isolating vertex i;
- F_{on} the set of vertices that must be on-tour, where $\{0, n+1\} \in F_{\text{on}}$ and $F_{\text{on}} \subseteq V$;
- F_{off} the set of vertices that must be off-tour and allocated, where $F_{\text{on}} \cap F_{\text{off}} = \varnothing$ and $F_{\text{off}} \subseteq V$. Any vertices that must be off-tour and isolated can be directly eliminated from the problem;
- λ_{tour} the objective function weight associated with tour costs;
- λ_{alloc} the objective function weight associated with allocation costs;
- λ_{isol} the objective function weight associated with isolation costs.

Then our binary decision variables are

- $x_{ij}=1$, if arc (i,j) is used as a tour arc, 0 otherwise;
- $y_{ij}=1$, if (off-tour) vertex j is allocated to (on-tour) vertex i, 0 otherwise;
- $v_i=1$, if vertex i is off-tour and isolated, 0 otherwise;
- $w_i=1$, if vertex i is off-tour and allocated to an on-tour vertex, 0 otherwise;
- $z_i=1$, if vertex i is on-tour, 0 otherwise.

So the formulation of SVRAP is

$$\left(\text{minimize } \lambda_{\text{tour}} \sum_{i=0}^{n} \sum_{j=1}^{n+1} c_{ij} x_{ij} + \lambda_{\text{alloc}} \sum_{i=0}^{n+1} \sum_{j=1}^{n} d_{ij} y_{ij} + \lambda_{\text{isol}} \sum_{i=1}^{n} D_i v_i \right) \tag{1}$$

subject to

$$v_i + w_i + z_i = 1, \quad \forall i \in V \tag{2}$$

$$z_i = 1, \quad \forall i \in F_{\text{on}} \tag{3}$$

$$w_i = 1, \quad \forall i \in F_{\text{off}} \tag{4}$$

$$\sum_{j=1}^{n+1} x_{0j} = 1 \tag{5}$$

$$\sum_{i=0}^{n} x_{i(n+1)} = 1 \tag{6}$$

$$\sum_{j=1}^{n+1} x_{ij} = \sum_{j=0}^{n} x_{ji} = z_i, \quad i = 1, \ldots, n \tag{7}$$

$$\sum_{i=0}^{n+1} y_{ij} = w_j, \quad j = 1, \ldots, n \tag{8}$$

$$y_{ij} \leqslant z_i, \quad \forall i \in V, \quad j = 1, \ldots, n \tag{9}$$

$$\sum_{j \in S, k \notin S} x_{jk} \geqslant z_i, \quad \forall i \in S, \quad \forall S \subset V \backslash \{n+1\}, \quad S \neq \varnothing \tag{10}$$

$$v_i, w_i, z_i \in \{0, 1\}, \quad \forall i \in V \tag{11}$$

$$x_{ij}, y_{ij} \in \{0, 1\}, \quad \forall i, j \in V \tag{12}$$

Equation (1) is a weighted sum of tour, allocation and isolation costs. Equation (2) ensures that each vertex is either off-tour (isolated or allocated) or on-tour. Equations (3) and (4) ensure that vertices in F_{on} are on-tour and that vertices in F_{off} are off-tour and allocated. Equations (5) and (6) ensure that the tour starts at 0 and ends at $n+1$. Equation (7) ensures that on-tour vertices have one incoming and one outgoing arc, and that off-tour vertices have no incoming or outgoing arcs. Equation (8) ensures that vertex j is allocated to exactly one other vertex if it is off-tour and allocated, and to no vertex otherwise. Equation (9) ensures that off-tour vertices are only allocated to on-tour vertices. Equation (10) is the subtour elimination constraint, which ensures that all appropriate sets S that contain an on-tour vertex have at least one leaving arc.

The weighting factors presented in Equation (1) reflect the fact that, in a practical application, the different costs involved are incurred by different parties. For example, if the vehicle is a mobile clinic then the tour costs are incurred by the clinic operator. Off-tour allocation costs are incurred by

those who travel (eg by public transport) to a visited vertex to use the clinic. Both of these costs can be considered as direct costs. Isolation costs are incurred by those who decide not to travel. As such they incur an opportunity cost, for example, the cost associated with the medical treatment that could have been avoided had they visited the clinic and received an early diagnosis.

Mathematically we could, if we wish, eliminate the isolation costs (and variables) from the above by allowing allocation to one of the vertices in F_{on}, say 0, to represent isolation, for example, set $d_{0j} = \min[d_{0j}, \lambda_{isol} D_j / \lambda_{alloc}]$, $j = 1, \ldots, n$. While the above formulation is an open tour formulation, if we wish to start and end at the same vertex (so a closed tour) then we simply create an artificial copy of that vertex. Note here that, in common with other approaches in the literature, we have assumed in the above formulation that the starting point for the tour (vertex 0) is known.

Literature survey

SVRAP as formulated above was first presented by Beasley and Nascimento (1996). They present no computational results, but discuss the links between SVRAP and other problems encountered in the literature. They distinguish between:

- special cases of SVRAP, no changes to the formulation required, only the setting of appropriate values for c_{ij}, d_{ij}, D_i and the weighting factors λ_{tour}, λ_{alloc}, λ_{isol},
- variants of SVRAP, where (slight) changes to the formulation are required.

Important special cases of SVRAP are

- Travelling salesman problem (Lawler *et al*, 1985), ($F_{on} = V$).
- Covering tour problem (Gendreau *et al*, 1997; Hodgson *et al*, 1998), where some vertices (F_{on}) must be on-tour and some vertices (F_{off}) must be off-tour and no more than a certain distance (Δ) from an on-tour vertex $\lambda_{tour} = \lambda_{alloc} = \lambda_{isol} = 1$, $D_i = 0$, $i = 1, \ldots, n$ and $d_{ij} = \infty$ if j is further from i than Δ, $d_{ij} = 0$ otherwise.
- Covering salesman problem (Current and Schilling, 1989), where every vertex either has to be on-tour or no more than a certain distance (Δ) from an on-tour vertex (so no isolated vertices allowed) $\lambda_{tour} = \lambda_{alloc} = \lambda_{isol} = 1$, $D_i = \infty$, $i = 1, \ldots, n$ and $d_{ij} = \infty$, if j further from i than Δ, $d_{ij} = 0$ otherwise. The open tour version of this problem is known as the shortest covering path problem (Current *et al*, 1984; Current *et al*, 1994).
- A variant (Bienstock *et al*, 1993) of the prize collecting travelling salesman problem (Balas, 1989), where a penalty is incurred for every vertex not visited ($\lambda_{tour} = \lambda_{isol} = 1$, $d_{ij} = \infty$, $\forall i, j$ and D_i represents the penalty for vertex i not being visited).

An early paper related to SVRAP is (Labbe and Laporte, 1986), dealing with post box location. Although their zero-one formulation is specific for post box location, it can be viewed (in part) as SVRAP with $\lambda_{tour} + \lambda_{alloc} = 1$ and no isolated vertices allowed. They refer to the problem as a location-allocation-routing problem.

Akinc and Srikanth (1992) relate the problem to the routing of a mobile service facility (where $\lambda_{tour} = \lambda_{alloc}$ and no isolated vertices are allowed). They present a branch and bound algorithm using a bound, involving solution of a directed minimal spanning tree problem, derived from Lagrangean relaxation. They also present a heuristic procedure based upon the solution to the Lagrangean problem. Computational results are presented for randomly generated problems involving up to 100 customers.

Peréz *et al* (2003) refer to the problem as the median cycle problem (where $\lambda_{tour} = \lambda_{alloc}$ and no isolated vertices are allowed) and present a heuristic for the problem based on variable neighbourhood tabu search, a combination of variable neighbourhood search (Mladenović and Hansen, 1997) and tabu search (Glover and Laguna, 1993, 1997; Gendreau, 2003). The moves that they consider relate to adding, or deleting, vertices from the tour. When a local minimum is reached classical TSP heuristics (two-optimal, three-optimal or Lin–Kernighan (Lin and Kernighan, 1973; Johnson and McGeoch, 2002) are applied. A shake procedure that consists of several random moves is also performed to escape local minima. Computational results are presented for problems involving up to 200 vertices.

Note here that the terminology adopted in the literature with respect to the phrase 'median cycle problem' is not uniquely defined. Peréz *et al* (2003), consider two problem variants—one as we have discussed above, the other where there exists a constraint upon the total allocation cost. Labbé *et al* (2005) consider only the second of these variants.

Labbé *et al* (2004) refer to the problem as the ring star problem (where $\lambda_{tour} = \lambda_{alloc}$ and no isolated vertices are allowed) and present a branch-and-cut algorithm for the problem. Their algorithm includes a heuristic procedure based upon the linear programming relaxation solution. Computational results are presented for a large number of problems involving up to 200 vertices.

Renaud *et al* (2004) present two heuristics for the median cycle problem. One is a multistart greedy add heuristic based on adding/removing vertices from the tour together with tour improvement using the heuristic given by Renaud *et al* (1996) as well as the Lin–Kernighan implementation of Helsgaun (Helsgaun, 2000). The other is a random keys evolutionary algorithm that uses solutions produced by their multistart greedy add heuristic. This algorithm also makes use of the Lin–Kernighan TSP heuristic. Computational results are presented for problems involving up to 200 vertices.

Tabu search algorithm

In this section, we describe our tabu search algorithm (TS-SVRAP) for SVRAP. We shall assume throughout this section familiarity with tabu search on the part of the reader. Further information with regard to tabu search can be found in (Glover and Laguna, 1993, 1997; Gendreau, 2003).

Moves

Let T represent the ordered set of on-tour vertices and S represent the set of off-tour vertices at any stage in TS-SVRAP. In SVRAP, decisions with respect to any off-tour vertex $j \in S$ can be made optimally in polynomial time simply by comparing the cost of isolating that vertex, $\lambda_{\text{isol}} D_j$, to the best allocation cost, $\min(\lambda_{\text{alloc}} d_{ij} | i \in T)$. In TS-SVRAP, at any stage, we always have every off-tour vertex optimally allocated/isolated. Given this, the problem reduces to one of deciding the set of vertices that are to be on-tour, and the tour to adopt through them. Note here that optimal allocation/isolation decisions for off-tour vertices depend only on the set of on-tour vertices, and are independent of the tour adopted through such vertices.

In order to simplify our description of TS-SVRAP, we shall henceforth assume that (c_{ij}) is symmetric, so that we are dealing with undirected edges in terms of the tour rather than directed arcs. The moves that we use in TS-SVRAP are

- ADD—adding vertex $i \in S \backslash F_{\text{off}}$ to T, where i is inserted between the two consecutive on-tour vertices that minimize the increase in tour cost, that is, i is inserted between j and k, where $c_{ji} + c_{ik} - c_{jk}$ minimizes $(c_{ai} + c_{ib} - c_{ab} | a, b$ consecutive vertices in T (so adjacent on the tour)).
- DROP—removing vertex $i \in T \backslash F_{\text{on}}$ and adding it to S, where the two vertices adjacent to i on the tour are connected together after i has been removed.
- TWOOPT—the usual TSP two-optimal tour improvement move, for example, see Johnson and McGeoch, 2002.

As commented above, after an ADD or DROP move any off-tour vertices are optimally allocated/isolated. Computationally an ADD move can be evaluated and performed in $O(n)$ operations, but a DROP move requires (in the worse case) $O(n^2)$ operations—since after a DROP move we may need to reallocate all off-tour vertices that were previously allocated to the vertex dropped from the tour. The TWOPT move can be evaluated and performed in constant time. Note here that, with respect to the tabu search heuristic presented previously by Peréz *et al* (2003), we have the same ADD and DROP moves, but they do not have a direct tour improvement move, rather they have an interchange move that swaps two vertices on/off-tour.

Initial solution

Figure 2 shows the optimal solution for a particular (closed tour, Euclidean) problem (eil51 with $\alpha = 7$) dealt with later. It can be seen there that there are a considerable number of vertices off-tour. It is also clear from Figure 2 that, for this Euclidean example, the tour does not visit vertices near to the border, or to the centre, of the domain.

We developed an initial solution procedure so as to try and generate initial solutions of the form shown in Figure 2. In order to obtain an estimate of the proportion of vertices on-tour, we constructed a single solution by creating a tour of all vertices (excluding those in F_{off}) using the nearest neighbour TSP heuristic (eg see Johnson and McGeoch, 2002) and then performed greedy local search, searching for improving TWOOPT, DROP and ADD moves (in that order, and making the first improving move encountered) until a local optimum was reached. Let P be the proportion of customer vertices on-tour in this solution.

Consider the rescaled (Euclidean) problem as shown in Figure 3, where we have rescaled the problem such that all vertices lie within the unit square. The intention is that vertices which fall into the shaded area will be forced on-tour in the initial solution, those outside the shaded area will be off-tour in that solution. The parameters a and b in Figure 3 were chosen by setting the width of the inner square equal to a (so giving $b = (1-a)/2$) and setting the area of the shaded region within the unit square to P (so that $(1-2a)^2 - (1-2b)^2 = P$). Simple algebra yields $a = (2 - \sqrt{1 + 3P})/3$. The initial solution was then composed of the nearest neighbour tour of all vertices in the shaded region (plus F_{on}), with all other vertices being off-tour. If a problem does not have an Euclidean interpretation

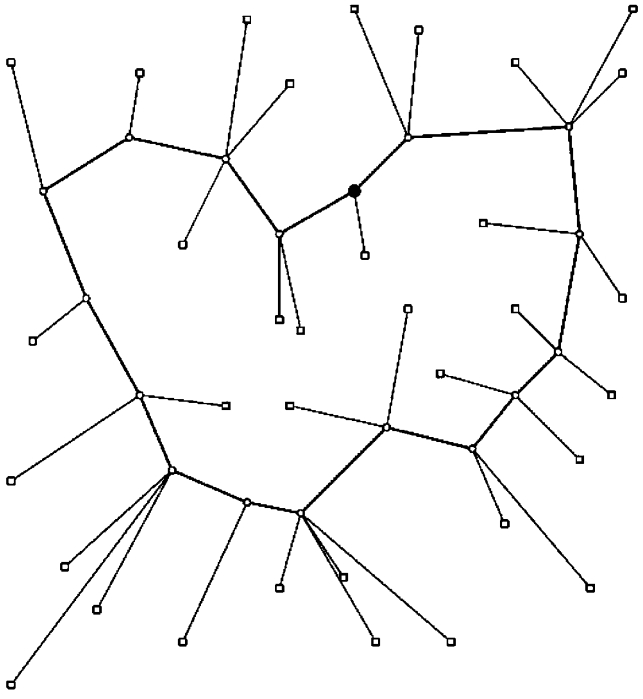

Figure 2 Optimal solution for an example problem.

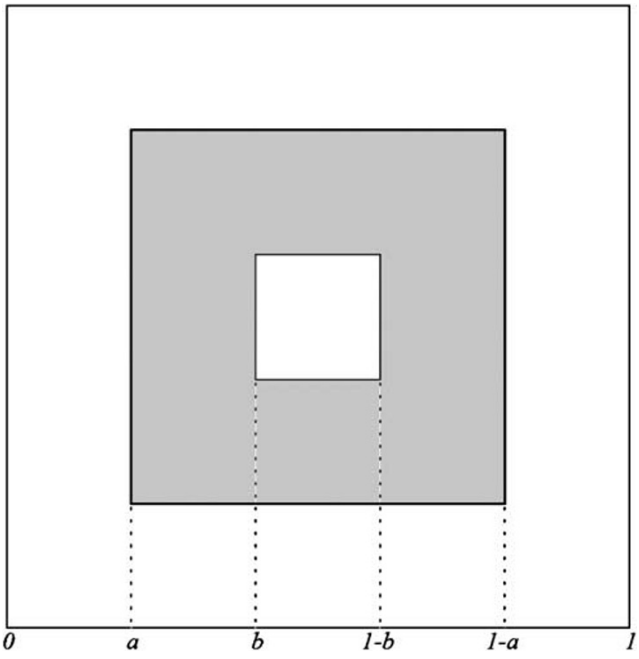

Figure 3 Initial solution generation.

then we simply take as the initial feasible solution the tour of all vertices (excluding those in F_{off}) found by using the nearest neighbour heuristic.

Note here that during the course of our tabu search algorithm TS-SVRAP we proceed from this initial feasible solution, and always remain feasible, that is, our algorithm operates purely in the space of feasible solutions, and never considers any infeasible solutions.

Tabu and aspiration

We have three different moves, ADD, DROP and TWOOPT. Since ADD and DROP can be regarded as the reverse of each other we had two separate tabu lists, one for ADD/DROP and one for TWOOPT, with each list having a different tabu tenure. In the computational results reported later we used a tabu tenure of 15 for ADD/DROP moves (so a move could not be reversed until 15 other ADD/DROP moves had been made), and a tabu tenure of 10 for TWOOPT moves (so a move could not be reversed until 10 other TWOOPT moves had been made).

In our algorithm, TS-SVRAP, we adopted a simple aspiration criterion and ignored the tabu status of a move if it led to a solution better than the previously best-known solution.

Path relinking

In TS-SVRAP, we adopted path relinking (Glover *et al*, 2000, 2003). This can be regarded as a procedure to intensify

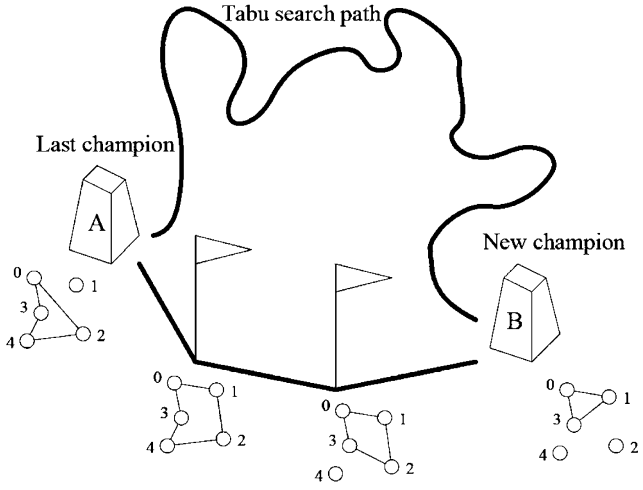

Figure 4 Path relinking.

the search in the region 'around' the current solution using information based on the previous best solution found.

To ease our discussion of path relinking we call a solution a '*champion*' if it is better than the previous champion (which was in turn better than all previous solutions) at given points in the search. In general in any local search algorithm (whether based on tabu search or some other approach) one traces a path in the solution space from one champion to another. Clearly the path in solution space between any two consecutive champions may be a long and indirect one. The idea of path relinking is to trace out a more 'direct' path between these two champions.

Figure 4 illustrates the idea of path relinking in the context of SVRAP for a closed tour problem (starting and ending at vertex 0) involving just a few vertices. In that figure, the newly discovered champion is B, and the previous champion is A. The solution associated with B, for example, is shown in that figure and consists of the (closed) tour 0-1-3, with vertices 2 and 4 being isolated. Comparing the solutions associated with B and A as shown in Figure 4, and regarding a vertex as being in one of two states (either on-tour or off-tour (allocated or isolated)) we can see that some vertices (0 and 3) have the same state in both solutions, others (1, 2 and 4) have changed state. We need to trace a more direct path between A and B than the path actually followed by the tabu search algorithm. In tracing this direct path we concentrate solely on the vertices that have a different state in A and B (so here vertices 1, 2 and 4). Vertices in the same state in each champion solution will not be changed in our path relinking process, and note here that this must by definition include any vertices in F_{on} or F_{off}.

In order to explain our implementation of path relinking for TS-SVRAP, let $\mu_{on}(i)$ be the percentage of champion solutions in which vertex i is on-tour and $\mu_{off}(i)$ be the percentage of champion solutions in which vertex i is off-tour (allocated or isolated). Suppose for the situation shown

Table 1 Values for path relinking example

Vertex i	*1*	*2*	*4*
$\mu_{on}(i)$	65	90	70
$\mu_{off}(i)$	35	10	30
State in champion solution A	Off-tour	On-tour	On-tour
State in champion solution B	On-tour	Off-tour	Off-tour
$\mu(i)$	65	10	30

in Figure 4 above, the values for $\mu_{off}(i)$ and $\mu_{on}(i)$ are as shown in Table 1, where we have only shown the vertices that are in a different state in A and B. So in Table 1, for example, vertex 1 has been on-tour in 65% of the champion solutions and off-tour in 35%. Let $\mu(i)$ be the percentage of champion solutions, where vertex i has been in the same state as in the current champion solution B. Then the $\mu(i)$ values (for vertices 1, 2 and 4, the only vertices of interest here in relation to path relinking) will be as shown in Table 1.

We use the $\mu(i)$ values to decide the order in which to change vertex states to trace out a more direct path from A to B, changing vertex states by considering vertices in descending $\mu(i)$ order, so here the vertices will be considered in order 1, 4 and 2. The solutions encountered in the relinked path from A to B can be seen in Figure 4:

- First vertex 1 is moved from off-tour (its state in A) to on-tour (its state in B)—this is accomplished by an ADD move (and recall here that the nature of an ADD move is such that the position in the tour for the added vertex is decided as part of the move);
- then vertex 4 is moved from on-tour (its state in A) to off-tour (its state in B)—this is accomplished by a DROP move (and recall here that the nature of a DROP move is such that the tour remaining after removing the vertex is decided as part of the move);
- and finally vertex 2 is moved from on-tour to off-tour—this is accomplished by a DROP move.

In other words, in moving from A to B we perform a sequence of ADD/DROP moves. Note here that, in Figure 4, in order to simplify the situation we have assumed that the optimal decision for any off-tour vertex is for it to be isolated. Obviously, in reality, we might well have a mix of allocated and isolated off-tour vertices.

The logic for considering vertex 1 first in tracing the path from A to B is that in the new champion solution B it is on-tour (it is off-tour in A); it has been on-tour in the highest percentage of champion solutions (across all vertices considered); and so making it on-tour is the most important state change. Similar logic applies to vertices 4 and 2, respectively.

Note here that if, after completing examination of all solutions on the path from A to B traced out by the relinking procedure described above, we find an improved solution then we update the champion solution appropriately. All ADD/DROP moves made during path relinking have their reverse moves set tabu, to prevent possible retracing of the path when we perform tabu search continuing from the champion solution.

Diversification

It is common in tabu search to implement some form of diversification procedure to enable the search process to explore different parts of the solution space. The two essential choices to be made relate to when diversification is to be done, and the starting point for the search after diversification.

In our TS-SVRAP algorithm, diversification was triggered once sufficient moves had been made from the current champion without discovering a new champion.

The solution from which we diversified (to find a new starting point in the solution space) was the current champion solution. We diversified the search by

- dropping from the champion tour vertices that have most frequently been on-tour (so a DROP move); and
- adding to the champion tour vertices that have most frequently been off-tour (so an ADD move).

In our implementation, we diversified by considering vertices in descending $\mu(i)$ order (excluding any vertices in F_{on} or F_{off}) and reversing their state. Diversification of this type can be viewed as frequency-based diversification—since we are making use of frequency information as to the state of the vertices at selected points (namely at champion solutions) in the search space.

Limited computational experience indicated that diversifying once 20 ADD/DROP, or 15 TWOOPT, moves had been made without discovery of a new champion was appropriate where $n/2$ vertices were diversified. Prior to diversification we cleared all tabu lists and made tabu the reverse of any diversification move made.

Complete tabu search algorithm

Our complete tabu search algorithm is shown in pseudocode in Figure 5. It can be seen there that we create the first champion (the starting point for the tabu search phase) using our area-based procedure followed by greedy local search (Steps 1–3).

In the tabu search phase (Step 4) we, in Step 4a, make those improving moves we can subject to move tabu status, while making any moves that lead to a new champion (irrespective of their tabu status—aspiration). At Step 4b, if we have improved upon the champion solution, then we perform path relinking. Step 4c diversifies the search if sufficient moves have been made since the current champion solution was found. In the event that neither path relinking or diversification are performed, Step 4d continues the

1. Create an initial solution using the area based approach.
2. Greedy local search. Repeat until no further move can be made: examine all TWOOPT, DROP and ADD moves (in that order) and make any improving move found.
3. Set the first champion equal to the locally optimum solution found at step 2.
4. Tabu search. Repeat until termination:
   a. Repeat until no further move can be made: examine all TWOOPT, DROP and ADD moves (in that order) and make any improving move that is not tabu, or is tabu but leads to a new champion. Tabu the reverse of any move made.
   b. If the current solution is better than the champion then update the champion and do the path relinking procedure. Update the champion solution in the event that path relinking finds one or more improved solutions.
   c. If sufficient moves (ADD/DROP or TWOOPT) have been made since the current champion was found then diversify the search.
   d. If neither path relinking nor diversification has been performed, then make the best non-improving move (either ADD/DROP or TWOOPT) that is not tabu and tabu the reverse of the move made.
5. Report the champion as the best solution found.

Figure 5 Pseudocode for TS-SVRAP.

search by making the best non-improving move (which computationally can be identified at Step 4a), making the reverse of that move tabu. Steps 4a–d are repeated until termination. In the computational results reported later, the termination criterion adopted was to stop once two diversifications had been performed.

As outlined in Figure 5 our heuristic, TS-SVRAP, involves no stochastic elements and so would produce exactly the same result at each replication. In order to introduce a stochastic element, and hence enable us to perform a number of replications we, for the purposes of creating the nearest neighbour solution (see above) perturbed the tour costs c_{ij} using $c_{ij} = c_{ij}(0.8 + 0.4r)$, where r is a random number drawn from $U(0, 1)$.

Computational results

In this section, we present computational results for our algorithm, TS-SVRAP, on a variety of test problems considered previously in the literature. We also present results for much larger test problems than have been considered by other researchers.

The tabu search algorithm, TS-SVRAP, presented above was programmed in C++ and run on a PC (Dell, 2.1 GB memory, 3.06 GHz Xeon processor) for all the problems used previously in (Labbé *et al*, 2004). These problems are derived from TSPLIB (Reinelt, 1991; http://www.iwr.uni-heidelberg.de/groups/comopt/software/TSPLIB95/ accessed June 2005) problems, where $c_{ij} = \alpha l_{ij}$ and $d_{ij} = (10-\alpha)l_{ij}$ (α takes the values 3,5,7,9 and l_{ij} is the distance given by the TSPLIB specification); $D_i = \infty$, $i = 1, \ldots, n$ (ie no isolated vertices allowed) and $\lambda_{\text{tour}} = \lambda_{\text{alloc}} = \lambda_{\text{isol}} = 1$. For these test problems as α increases, the percentage of vertices on-tour decreases. The depot (vertex 0) is the first vertex in the TSPLIB file (so the number of customers n is one less than the number of vertices in the TSPLIB file) and the tour is closed by identifying the end point with the depot.

Our algorithm requires a variety of parameter values to be specified (eg tabu tenure, number of moves before diversification) and for the purposes of deciding such parameter values we took a small subset (specifically eil51, rd100, ch150 and kroA200) of the problems used in (Labbé *et al*, 2004), and decided parameter values by examining different choices on this subset. These parameter values were decided singly (ie in a sequential fashion by varying one parameter at a time) as opposed to making a simultaneous choice of parameter values by varying many parameters simultaneously.

We performed 500 replications of our algorithm for each problem and Table 2 presents results on the 132 problems considered by Labbé *et al* (2004). In that table we, for each problem solved, present:

- the solution given by Labbé *et al* (2004) and the time reported (h:min:s on a Pentium III, 866 MHz),
- the best solution found by our algorithm (over all replications), the percentage deviation from the solution found by Labbé *et al* (2004), (100(our solution—Labbé solution)/Labbé solution), the replication at which the best solution found was first encountered, the percentage of customer vertices on-tour/allocated in that solution and the total time taken over all replications (seconds).

The algorithm presented by Labbé *et al* (2004) is an optimal procedure and the majority of the solutions found by them are optimal. For some test problems they terminated their algorithm after 2 h of computation and these problems are indicated in Table 2 by the solution value (the best solution they found before termination) being given in brackets. Note that for pr76 with $\alpha = 7$ we obtain a better result than the optimum reported by Labbé *et al* (2004). Since this occurs only once, and since for small problems we often get the same result as given by Labbé *et al* (2004), we believe that the optimum value reported by Labbé *et al* (2004) for this problem is incorrect.

None of the problems considered by Labbé *et al* (2004) allowed isolated vertices. In order to investigate the performance of our tabu search algorithm when vertices can be isolated we also present in Table 2 results for these problems, but with isolation costs defined by $D_i = \min(d_{ij} | j \neq i, j \in V)$, $i = 1, \ldots, n$. Limited computational experience indicated that in order to generate solutions with

Table 2 Results for Labbé *et al*, 2004, problems

		Problems with no isolated vertices allowed								*Problems with isolated vertices allowed*					
				Tabu search TS-SVRAP						*Tabu search TS-SVRAP*					
		Labbé et al (2004) results					*Percentage of vertices*					*Percentage of vertices*			
Problem	α	*Solution*	*Time (h:min:s)*	*Solution*	*Percentage deviation*	*Replication*	*On-tour*	*Allocated*	*Total time (s)*	*Solution*	*Replication*	*On-tour*	*Allocated*	*Isolated*	*Total time (s)*
eil51	3	1278	0:00:02	1278	0	408	100.0	0.0	4.6	1274.72	50	92.0	0.0	8.0	2.8
	5	1995	0:00:01	1995	0	47	76.0	24.0	2.4	1710.00	217	0.0	6.0	94.0	3.6
	7	2113	0:00:10	2113	0	2	32.0	68.0	1.9	1480.32	2	0.0	10.0	90.0	2.5
	9	1244	0:00:10	1244	0	1	10.0	90.0	1.5	685.28	77	0.0	12.0	88.0	2.4
berlin52	3	22626	0:00:01	22626	0	59	100.0	0.0	4.1	21733.31	5	90.2	0.0	9.8	3.4
	5	36115	0:00:04	36115	0	37	78.4	21.6	5.0	26520.30	208	0.0	2.0	98.0	4.1
	7	37376	0:00:06	37376	0	15	45.1	54.9	2.3	23525.81	3	0.0	3.9	96.1	2.6
	9	20361	0:00:12	20361	0	1	7.8	92.2	2.0	11508.59	1	2.0	21.6	76.5	2.5
brazil58	3	76185	0:00:02	76185	0	1	100.0	0.0	5.9	66929.29	2	91.2	0.0	8.8	3.2
	5	115045	0:00:14	115045	0	15	68.4	31.6	5.6	78553.15	2	1.8	0.0	98.2	5.6
	7	126807	0:00:12	126807	0	1	47.4	52.6	2.1	69372.22	1	0.0	1.8	98.2	4.2
	9	83690	0:00:28	83690	0	1	14.0	86.0	3.1	33418.00	1	0.0	1.8	98.2	3.0
st70	3	2025	0:00:13	2031	0.30	56	100.0	0.0	8.8	1987.48	236	82.6	0.0	17.4	7.8
	5	3110	0:00:24	3110	0	11	73.9	26.1	4.7	2560.20	64	1.4	1.4	97.1	5.9
	7	3402	0:00:20	3402	0	2	42.0	58.0	3.1	2344.45	1	0.0	4.3	95.7	5.4
	9	2610	0:01:10	2610	0	1	23.2	76.8	2.9	1137.30	2	0.0	7.2	92.8	5.1
eil76	3	1614	0:00:05	1614	0	239	100.0	0.0	10.3	1620.00	27	100.0	0.0	0.0	8.6
	5	2460	0:00:21	2460	0	12	73.3	26.7	6.0	2446.25	418	62.7	21.3	16.0	7.0
	7	2504	0:00:37	2504	0	14	41.3	58.7	4.4	2417.01	17	32.0	38.7	29.3	5.0
	9	1710	0:02:28	1710	0	5	14.7	85.3	4.0	1162.27	96	0.0	17.3	82.7	6.3
pr76	3	324477	0:17:39	324912	0.13	49	100.0	0.0	11.4	300716.45	126	78.7	0.0	21.3	8.9
	5	500395	0:02:27	500415	0.00	60	82.7	17.3	8.0	405873.75	9	1.3	0.0	98.7	6.8
	7	555858	0:01:54	555845	−0.00	24	50.7	49.3	5.4	386198.38	7	1.3	1.3	97.3	6.7
	9	424359	0:03:04	424359	0	10	16.0	84.0	3.6	183882.74	3	0.0	2.7	97.3	5.3
gr96	3	163935	0:01:43	167124	1.95	14	100.0	0.0	12.0	164772.07	119	93.7	0.0	6.3	14.0
	5	252850	0:01:06	255615	1.09	451	76.8	23.2	11.1	249510.75	296	65.3	16.8	17.9	12.8
	7	275599	0:01:08	276270	0.24	166	51.6	48.4	9.7	269022.34	200	47.4	42.1	10.5	9.6
	9	232823	0:06:53	235287	1.06	48	27.4	72.6	10.0	134619.21	1	1.1	10.5	88.4	9.9
rat99	3	3633	0:00:38	3636	0.08	147	100.0	0.0	14.2	3629.70	147	98.0	0.0	2.0	13.9
	5	5885	0:00:35	5895	0.17	20	86.7	13.3	9.7	5865.80	303	78.6	14.3	7.1	10.9
	7	6436	0:01:20	6436	0	267	41.8	58.2	7.8	6280.62	10	37.8	50.0	12.2	9.6
	9	5150	0:07:19	5167	0.33	343	19.4	80.6	5.8	3524.52	2	0.0	0.0	100.0	9.3
kroA100	3	63846	0:00:37	63888	0.07	75	100.0	0.0	19.0	63662.28	423	97.0	0.0	3.0	17.6
	5	100785	0:01:26	100785	0	59	79.8	20.2	12.5	96432.55	66	58.6	16.2	25.3	9.7
	7	115388	0:01:16	115388	0	11	54.5	45.5	10.8	98970.35	96	0.0	6.1	93.9	10.6
	9	94265	0:05:57	94467	0.21	33	21.2	78.8	7.0	50711.47	2	0.0	1.0	99.0	9.9
kroB100	3	66423	0:01:38	66801	0.57	319	100.0	0.0	19.3	64292.48	120	79.8	0.0	20.2	21.8
	5	104550	0:00:57	104550	0	262	76.8	23.2	14.2	97005.90	27	59.6	18.2	22.2	12.6
	7	118111	0:01:17	118363	0.21	166	46.5	53.5	8.3	101410.40	1	0.0	6.1	93.9	7.9
	9	93938	0:07:34	94026	0.09	3	15.2	84.8	7.9	50165.39	1	0.0	9.1	90.9	11.0

Table 2 Continued

		Problems with no isolated vertices allowed								*Problems with isolated vertices allowed*					
				Tabu search TS-SVRAP						*Tabu search TS-SVRAP*					
		Labbé et al (2004) results					*Percentage of vertices*					*Percentage of vertices*			
Problem	α	*Solution*	*Time (h:min:s)*	*Solution*	*Percentage deviation*	*Replication*	*On-tour*	*Allocated*	*Total time (s)*	*Solution*	*Replication*	*On-tour*	*Allocated*	*Isolated*	*Total time (s)*
kroC100	3	62247	0:00:33	62259	0.02	44	100.0	0.0	18.7	61947.96	397	91.9	0.0	8.1	18.5
	5	99065	0:00:45	99065	0	220	82.8	17.2	10.4	96380.00	181	66.7	13.1	20.2	12.1
	7	113533	0:01:07	113533	0	27	49.5	50.5	9.7	105265.89	167	15.2	16.2	68.7	10.8
	9	92894	0:07:05	93358	0.50	4	22.2	77.8	5.2	49107.86	1	0.0	9.1	90.9	10.5
kroD100	3	63882	0:00:32	64083	0.31	53	100.0	0.0	21.8	62731.34	62	89.9	0.0	10.1	18.2
	5	101645	0:00:38	101655	0.01	186	80.8	19.2	15.4	96992.15	270	48.5	13.1	38.4	14.0
	7	116849	0:01:50	116849	0	264	46.5	53.5	9.8	101397.45	394	1.0	0.0	99.0	8.8
	9	92102	0:06:58	92133	0.03	16	18.2	81.8	9.0	50578.72	3	0.0	4.0	96.0	9.8
kroE100	3	66204	0:01:44	66348	0.22	358	100.0	0.0	19.3	62821.45	189	75.8	0.0	24.2	21.8
	5	104915	0:01:54	104915	0	115	77.8	22.2	12.1	96772.60	37	53.5	10.1	36.4	11.6
	7	116471	0:01:06	116471	0	1	50.5	49.5	10.1	102200.14	418	1.0	8.1	90.9	8.6
	9	96116	0:08:25	96116	0	1	20.2	79.8	6.4	51060.57	1	1.0	13.1	85.9	10.3
rd100	3	23730	0:00:16	23739	0.04	219	100.0	0.0	19.4	23748.00	34	100.0	0.0	0.0	16.9
	5	37975	0:01:40	38045	0.18	98	75.8	24.2	11.1	37019.25	51	64.6	22.2	13.1	10.5
	7	40915	0:01:53	40924	0.02	29	41.4	58.6	8.0	39039.86	168	28.3	40.4	31.3	8.8
	9	31776	0:07:01	31795	0.06	63	20.2	79.8	6.5	18928.44	5	0.0	11.1	88.9	14.4
eil101	3	1887	0:00:46	1914	1.43	1	100.0	0.0	21.7	1905.00	381	100.0	0.0	0.0	21.2
	5	2905	0:00:31	2945	1.38	13	75.0	25.0	12.1	2905.00	4	71.0	29.0	0.0	12.6
	7	2926	0:04:00	2926	0	143	39.0	61.0	8.3	2919.08	28	37.0	59.0	4.0	8.7
	9	1955	0:08:18	1955	0	106	18.0	82.0	6.0	1727.12	3	0.0	26.0	74.0	11.2
lin105	3	43137	0:00:21	43563	0.99	171	100.0	0.0	14.7	40853.28	227	60.6	0.0	39.4	24.2
	5	69365	0:00:43	69365	0	65	79.8	20.2	10.1	57850.10	50	49.0	13.5	37.5	10.7
	7	83597	0:01:38	83751	0.18	288	53.8	46.2	11.8	56102.32	92	1.0	1.9	97.1	10.9
	9	69920	0:07:30	69920	0	2	30.8	69.2	9.0	28220.47	6	0.0	1.9	98.1	10.3
pr107	3	132909	0:00:23	132909	0	415	100.0	0.0	20.6	132909.00	415	100.0	0.0	0.0	19.2
	5	210465	0:00:50	210465	0	1	67.9	32.1	15.7	164542.40	12	7.5	2.8	89.6	10.8
	7	259571	0:01:52	260145	0.22	1	40.6	59.4	7.7	162933.06	72	0.0	1.9	98.1	11.2
	9	264918	0:06:45	265727	0.31	1	25.5	74.5	14.9	82645.08	72	0.0	1.9	98.1	9.5
gr120	3	20826	0:01:01	21003	0.85	455	100.0	0.0	28.8	20882.33	38	95.8	0.0	4.2	27.1
	5	31480	0:02:38	31595	0.37	478	73.1	26.9	21.0	30877.50	35	69.7	23.5	6.7	22.7
	7	32301	0:03:19	32306	0.02	133	42.0	58.0	16.8	31522.12	71	40.3	50.4	9.2	18.5
	9	24322	0:11:34	24322	0	29	21.8	78.2	11.3	20868.59	13	0.0	10.9	89.1	15.3
pr124	3	177090	0:12:32	177261	0.10	134	100.0	0.0	23.2	177261.00	134	100.0	0.0	0.0	17.2
	5	286115	0:03:58	287150	0.36	4	88.6	11.4	17.3	283210.20	190	74.0	4.9	21.1	17.5
	7	358853	0:03:27	360558	0.48	30	68.3	31.7	15.0	302742.67	325	12.2	5.7	82.1	14.0
	9	340153	0:16:05	344202	1.19	208	36.6	63.4	9.5	152500.03	452	0.0	5.7	94.3	13.4

Table 2 Continued

		Problems with no isolated vertices allowed								*Problems with isolated vertices allowed*					
				Tabu search TS-SVRAP						*Tabu search TS-SVRAP*					
		Labbé et al (2004) results					*Percentage of vertices*					*Percentage of vertices*			
Problem	α	*Solution*	*Time (h:min:s)*	*Solution*	*Percentage deviation*	*Replication*	*On-tour*	*Allocated*	*Total time (s)*	*Solution*	*Replication*	*On-tour*	*Allocated*	*Isolated*	*Total time (s)*
bier127	3	354846	0:02:27	357606	0.78	301	100.0	0.0	34.7	357841.63	302	96.0	0.0	4.0	34.1
	5	539955	0:03:32	543040	0.57	432	77.8	22.2	21.6	521368.80	219	66.7	19.8	13.5	22.3
	7	567110	0:07:11	568426	0.23	50	44.4	55.6	15.5	514651.85	384	34.1	42.9	23.0	17.3
	9	347845	0:36:52	348042	0.06	32	15.1	84.9	10.6	281832.35	2	9.5	55.6	34.9	17.1
ch130	3	18330	0:02:07	18636	1.67	92	100.0	0.0	35.1	17887.88	266	73.6	0.0	26.4	42.3
	5	28790	0:02:38	28825	0.12	3	80.6	19.4	25.7	26791.35	5	66.7	12.4	20.9	22.0
	7	32707	0:11:24	32762	0.17	286	45.0	55.0	16.7	29226.98	22	35.7	39.5	24.8	20.4
	9	23639	0:21:56	23639	0	174	15.5	84.5	13.8	16968.38	2	0.8	17.8	81.4	17.8
pr136	3	290316	0:15:04	295353	1.74	499	100.0	0.0	23.8	295239.68	30	98.5	0.0	1.5	21.3
	5	468520	0:02:23	469315	0.17	229	85.2	14.8	19.2	463325.00	438	71.1	11.1	17.8	21.4
	7	491981	0:06:01	493117	0.23	54	44.4	55.6	17.3	485614.52	247	34.1	57.0	8.9	25.0
	9	387327	0:25:33	387327	0	58	25.2	74.8	16.5	334836.43	5	0.0	14.1	85.9	21.7
gr137	3	208929	0:02:36	211581	1.27	470	100.0	0.0	32.1	211038.52	346	98.5	0.0	1.5	28.9
	5	329465	0:08:59	330295	0.25	78	82.4	17.6	23.2	327140.00	417	79.4	14.7	5.9	25.4
	7	366022	0:07:06	367663	0.45	110	55.1	44.9	17.5	365134.42	454	53.7	39.7	6.6	17.9
	9	335009	0:31:09	336151	0.34	11	25.7	74.3	15.6	225646.16	407	0.0	2.9	97.1	21.7
pr144	3	175611	0:02:35	175770	0.09	432	100.0	0.0	27.5	159898.68	390	65.0	0.0	35.0	44.9
	5	290945	0:51:01	291115	0.06	13	93.7	6.3	42.9	223135.00	59	47.6	4.9	47.6	30.1
	7	383041	0:07:22	383041	0	6	63.6	36.4	16.6	267356.13	2	32.2	11.2	56.6	20.5
	9	366833	0:25:44	366848	0.00	236	23.1	76.9	13.8	94220.10	1	0.7	3.5	95.8	23.8
ch150	3	19584	0:05:07	19680	0.49	22	100.0	0.0	47.4	19658.62	90	98.0	0.0	2.0	43.8
	5	31170	0:05:15	31180	0.03	66	77.2	22.8	27.8	31144.80	177	73.2	20.8	6.0	25.3
	7	34930	0:09:21	34930	0	234	47.7	52.3	23.4	34557.09	233	42.3	50.3	7.4	24.6
	9	26371	0:54:12	26479	0.41	159	18.1	81.9	19.0	24118.64	4	0.0	14.1	85.9	25.8
kroA150	3	79572	0:17:05	80571	1.26	128	100.0	0.0	47.3	79210.79	167	94.6	0.0	5.4	52.6
	5	125435	0:03:44	125620	0.15	419	77.2	22.8	31.3	123277.75	248	72.5	18.8	8.7	30.6
	7	140961	0:09:05	141093	0.09	313	49.7	50.3	20.0	134500.79	81	44.3	39.6	16.1	30.7
	9	113080	0:45:36	113177	0.09	99	19.5	80.5	14.4	83145.57	3	0.0	16.1	83.9	25.4
kroB150	3	78390	0:10:33	78963	0.73	442	100.0	0.0	51.9	78347.01	278	91.9	0.0	8.1	52.8
	5	122875	1:52:12	122995	0.10	45	77.9	22.1	27.1	120803.10	352	65.8	20.8	13.4	30.7
	7	135382	0:07:01	135680	0.22	64	53.7	46.3	23.4	131962.93	206	47.0	39.6	13.4	31.8
	9	108885	0:43:22	109143	0.24	37	18.1	81.9	16.9	84281.91	20	0.0	11.4	88.6	27.3
pr152	3	221046	0:09:42	222459	0.64	387	100.0	0.0	42.9	222459.00	387	100.0	0.0	0.0	28.1
	5	(376155)	2:00:00	364605	−3.07	95	92.1	7.9	37.2	302747.40	31	55.6	8.6	35.8	32.3
	7	(475052)	2:00:00	467431	−1.60	16	57.6	42.4	14.7	360259.42	461	30.5	25.8	43.7	21.9
	9	475440	0:52:15	477110	0.35	198	19.2	80.8	16.6	165691.53	1	0.0	3.3	96.7	22.3
u159	3	126240	0:02:27	127431	0.94	461	100.0	0.0	41.7	127431.00	461	100.0	0.0	0.0	34.0
	5	204250	0:09:08	204340	0.04	480	84.8	15.2	36.1	204280.00	108	81.6	18.4	0.0	34.0
	7	235221	0:09:24	235655	0.18	498	53.2	46.8	20.8	233005.01	57	52.5	44.3	3.2	27.0
	9	199552	0:59:50	199573	0.01	107	25.9	74.1	19.7	165499.63	10	0.0	5.7	94.3	26.9

Table 2 Continued

		Problems with no isolated vertices allowed								*Problems with isolated vertices allowed*					
				Tabu search TS-SVRAP						*Tabu search TS-SVRAP*					
		Labbé et al (2004) results					*Percentage of vertices*					*Percentage of vertices*			
Problem	α	*Solution*	*Time (h:min:s)*	*Solution*	*Percentage deviation*	*Replication*	*On-tour*	*Allocated*	*Total time (s)*	*Solution*	*Replication*	*On-tour*	*Allocated*	*Isolated*	*Total time (s)*
rat195	3	6969	0:29:41	7092	1.76	435	100.0	0.0	58.0	7092.00	435	100.0	0.0	0.0	60.5
	5	11320	0:12:21	11440	1.06	161	82.0	18.0	39.9	11415.00	325	85.1	14.9	0.0	41.5
	7	12319	0:34:25	12400	0.66	242	39.2	60.8	37.1	12402.00	66	41.8	58.2	0.0	35.9
	9	(9395)	2:00:00	9015	−4.04	177	17.0	83.0	25.4	9007.00	57	17.5	82.5	0.0	30.3
d198	3	47340	0:25:54	47688	0.74	412	100.0	0.0	80.2	47688.00	412	100.0	0.0	0.0	70.1
	5	76945	1:12:20	77560	0.80	320	85.3	14.7	46.7	78108.60	208	81.7	15.2	3.0	38.4
	7	94300	1:48:59	94789	0.52	105	51.8	48.2	56.1	84886.01	457	42.1	43.7	14.2	57.4
	9	(97899)	2:00:00	96241	−1.69	470	21.3	78.7	36.6	73800.87	406	14.7	50.8	34.5	44.9
kroA200	3	(93699)	2:00:00	88992	−5.02	466	100.0	0.0	90.3	87813.12	354	93.5	0.0	6.5	114.9
	5	(138885)	2:00:00	139385	0.36	235	74.9	25.1	54.1	138338.50	266	72.4	21.6	6.0	58.1
	7	158227	1:31:05	158940	0.45	401	47.7	52.3	49.6	153502.91	196	43.2	48.2	8.5	58.1
	9	(124678)	2:00:00	122833	−1.48	194	18.1	81.9	29.9	119310.82	75	16.1	61.8	22.1	48.0
kroB200	3	88311	0:13:44	89757	1.64	118	100.0	0.0	90.2	89037.11	271	96.0	0.0	4.0	106.6
	5	138905	0:25:44	140275	0.99	305	78.9	21.1	62.4	139657.90	108	75.9	20.6	3.5	63.9
	7	156638	0:25:19	159142	1.60	457	48.7	51.3	39.7	157029.57	332	38.7	44.2	17.1	41.0
	9	(127800)	2:00:00	124049	−2.94	2	17.6	82.4	34.4	114280.01	9	16.6	60.8	22.6	79.3
Average	3		0:09:05		0.48	239	100.0	0.0	30.3		222	91.6	0.0	8.4	30.9
	5		0:17:17		0.16	153	79.5	20.5	21.2		164	55.4	13.7	30.9	20.5
	7		0:14:37		0.14	134	48.1	51.9	15.8		160	25.1	28.2	46.7	18.0
	9		0:30:03		−0.15	86	20.1	79.9	12.4		53	2.4	17.2	80.4	18.2
Average			0:17:45		0.16	153	61.9	38.1	19.9		150	43.6	14.8	41.6	21.9

a reasonable mix of on-tour, allocated and isolated vertices suitable values for the objective function weighting parameters were $\lambda_{tour} = \lambda_{alloc} = 1$ and $\lambda_{isol} = 0.5 + 0.0004\alpha^2 n$. Table 2 shows the results obtained by our tabu search algorithm for these problems, where we show the percentage of customer vertices of each type (on-tour, allocated and isolated) in the final best solution found.

Considering Table 2 it is clear that, on average, our tabu search heuristic produces good-quality (near-optimal) solutions in reasonable computation times. Overall TS-SVRAP produces solutions 0.16% above those in (Labbé *et al*, 2004), in an average computation time of 19.9 s. As problem size increases, our algorithm also performs well. Of the eight test problem that were not solved to optimality in 2 h of computation time by Labbé *et al* (2004), our algorithm finds improved feasible solutions for seven problems (in an average computation time of 40.3 s). Considering the averages presented at the bottom of Table 2 for different α values it is clear that, in general, as α increases (ie the percentage of vertices on-tour decreases) computation time for TS-SVRAP decreases.

Of the 132 problems in Table 2 considered by Labbé *et al* (2004), a subset of 52 problems have been considered by Renaud *et al* (2004) and Peréz *et al* (2003). Rather than giving detailed results for each test problem for each of their approaches Table 3 summarizes the results obtained. That table is drawn from our results in Table 2 and from the results presented by Renaud *et al* (2004) (Tables 1 and 6).

In Table 3 results are shown for Labbé *et al*, 2004; multistart greedy add (MGA) (Renaud *et al*, 2004); random keys evolutionary algorithm (RKEA) (Renaud *et al*, 2004); variable neighbourhood tabu search (VNTS) (Peréz *et al*, 2003), taken from (Renaud *et al*, 2004); and our tabu search algorithm. All of these test problems have been solved optimally by Labbé *et al* (2004), and so we present the average percentage deviation from optimal for each algorithm.

One issue with respect to Table 3 is that Peréz *et al* (2003) present detailed results that are based on a version of Labbé *et al* (2004) that predates (and indeed appears to be different from) the version finally published. We have assumed here therefore that Renaud *et al* (2004), since Laporte is a co-author in both Renaud *et al* (2004) and Labbé *et al* (2004), have reinterpreted the results from Peréz *et al* (2003) appropriately.

Clearly one complication in trying to compare the results in Table 3 is the different computers used. Utilizing (Dongarra, 2005; http://www.netlib.org/benchmark/performance.ps accessed June 2005) we have made an *approximate* estimate of the equivalent times for each algorithm in Pentium III, 866 MHz, seconds, and this is also given in Table 3. On balance, and bearing in mind the approximate nature of the computational comparison made, it seems reasonable to conclude from Table 3 that

- MGA provides reasonable quality results very quickly,
- our tabu search algorithm, TS-SVRAP, provides better quality results than MGA, but at the expense of more computation time,
- VNTS is not competitive with our tabu search algorithm,
- RKEA takes longer (for these test problems) than solving them to proven optimality.

The results for our algorithm TS-SVRAP presented in Table 3 were obtained using a two-optimal procedure for deciding the vertex tour. This contrasts with VNTS (Peréz *et al*, 2003), and both MGA and RKEA (Renaud *et al*, 2004), which use more sophisticated tour improvement heuristics (three-optimal or Lin–Kernighan). In order to investigate the effect of a more sophisticated tour improvement heuristic in conjunction with TS-SVRAP, we applied a three-optimal procedure to the tour associated with the best solution found at each replication (so we did not change the on-tour/off-tour status of each vertex). This had only a slight effect on solution quality, improving the solution for just three of the 52 problems in Table 3 and reducing the average percentage deviation from optimal to 0.17%, but at the expense of increasing the computation time by 50%.

As far as we are aware, problems of the size shown in Table 2 are the largest problems considered to date in the literature (either for optimal or heuristic algorithms). In order to provide some benchmark results for future researchers, we have also applied our tabu search algorithm

Table 3 Summary results for a subset of 52 problems

	Labbé et al, 2004	*Renaud et al, 2004*		*Peréz et al, 2003 from Renaud et al, 2004*	*Tabu search*
		MGA	*RKEA*	*VNTS*	*TS-SVRAP*
Average percentage deviation from optimal	0	0.24	0.09	0.25	0.18
Average time (seconds)	542	12.5	730	377	13.7
Approximate average time (Pentium III, 866 MHz, seconds)	542	13	759	271	101

For MGA we have taken the lowest percentage deviation result presented in Renaud *et al*, 2004 over the 72 different parameter sets tried on all 52 test problems.
Computers used are: Labbé *et al*, Pentium III, 866 MHz; MGA and RKEA, Pentium III, 900 MHz; VNTS, Sun Ultra 60, 300 MHz; TS-SVRAP, Dell PC, Xeon 3.06 GHz.

Table 4 Results for larger test problems

Problem	α	*Solution*	*Replication*	*Percentage of vertices*		*Total time (s)*
				On-tour	*Allocated*	
d493	3	108939	422	100.0	0.0	347
	5	171945	183	84.3	15.7	502
	7	190938	437	46.3	53.7	725
	9	153348	124	18.5	81.5	432
rat783	3	27843	304	100.0	0.0	1145
	5	44250	238	78.8	21.2	1484
	7	48779	460	46.3	53.7	2071
	9	34639	289	18.8	81.2	1154
pr1002	3	817725	17	100.0	0.0	2147
	5	1282450	491	78.3	21.7	2803
	7	1427761	277	47.1	52.9	4560
	9	1057273	428	18.6	81.4	2273
fl1577	3	68979	300	100.0	0.0	6722
	5	111155	20	87.6	12.4	11742
	7	137313	14	55.0	45.0	17810
	9	129551	262	32.2	67.8	15735
u2152	3	205419	230	100.0	0.0	20531
	5	337440	361	90.5	9.5	14769
	7	385888	441	47.9	52.1	60034
	9	281718	322	19.9	80.1	32892
pcb3038	3	444681	406	100.0	0.0	53148
	5	701410	276	84.0	16.0	50482
	7	767092	152	45.4	54.6	139414
	9	542139	151	17.1	82.9	91640
rl5934	3	1783167	397	100.0	0.0	289481
	5	2897725	421	91.7	8.3	334756
	7	3544630	415	64.1	35.9	627142
	9	2936791	393	30.1	69.9	602614
Average	3		297	100.0	0.0	53360
	5		284	85.0	15.0	59505
	7		314	50.3	49.7	121679
	9		281	22.2	77.8	106677
Average			294	64.4	35.6	85306

to a set of larger problems, also drawn from TSPLIB and produced in the same manner as detailed in (Labbé *et al*, 2004), so no isolated vertices allowed, involving up to 5934 vertices. Results for these problems can be seen in Table 4.

In order to investigate the practical computational complexity of our algorithm we, using the results shown in Table 4, performed a least squares linear regression to get the best fit line of the form: $time\ taken = \beta n^{\gamma}$. The result was statistically highly significant, with the best fit line having $\beta = 10^{-4.9}$, $\gamma = 2.8$. Hence, the practical computational complexity of our algorithm is $O(n^{2.8})$.

Conclusions

In this paper, we have presented a tabu search algorithm for the single vehicle routing allocation problem. Our algorithm includes moves relating to changing the set of on-tour vertices, as well as a move related to changing the tour adopted around the on-tour vertices. It also includes aspiration, path relinking and frequency-based diversification.

Computational results were presented for a set of test problems used previously in the literature. These indicated that our algorithm produced good-quality results in a reasonable computation time. We also reported results for much larger problems than have been considered by other researchers.

Acknowledgements—The support offered by Professor TM Liebling (EPFL, Lausanne, Switzerland) that enabled L Vogt to visit Brunel University is gratefully acknowledged.

References

Akinc U and Srikanth K (1992). Optimal routing and process scheduling for a mobile service facility. *Networks* **22**: 163–183.

Balas E (1989). The prize collecting traveling salesman problem. *Networks* **19**: 621–636.

Beasley JE and Nascimento EM (1996). The vehicle routing-allocation problem: a unifying framework. *TOP* **4**: 65–86.

Bienstock D, Goemans MX, Simchi-Levi D and Williamson D (1993). A note on the prize collecting traveling salesman problem. *Math Program* **59**: 413–420.

Current J, Pirkul H and Rolland E (1994). Efficient algorithms for solving the shortest covering path problem. *Transport Sci* **28**: 317–327.

Current J, ReVelle C and Cohon J (1984). The shortest covering path problem: an application of locational constraints to network design. *J Region Sci* **24**: 161–183.

Current JR and Schilling DA (1989). The covering salesman problem. *Transport Sci* **23**: 208–213.

Dongarra JJ (2005). Performance of various computers using standard linear equations software. http://www.netlib.org/benchmark/performance.ps, accessed June 2005.

Gendreau M (2003). An introduction to tabu search. In: Glover F and Kochenberger GA (eds). *Handbook of Metaheuristics*. Kluwer Academic Publishers, Dordrecht, The Netherlands, pp 37–54.

Gendreau M, Laporte G and Semet F (1997). The covering tour problem. *Opns Res* **45**: 568–576.

Glover F and Laguna M (1993). Tabu search. In: Reeves CR (ed). *Modern Heuristic Techniques for Combinatorial Problems*. Blackwell Scientific Publications, Oxford, UK, pp 70–150.

Glover F and Laguna M (1997). *Tabu Search*. Kluwer Academic Publishers: Dordrecht, The Netherlands.

Glover F, Laguna M and Marti R (2000). Fundamentals of scatter search and path relinking. *Control and Cybernetics* **29**: 653–684.

Glover F, Laguna M and Marti R (2003). Scatter search and path relinking: advances and applications. In: Glover F and Kochenberger GA (eds). *Handbook of Metaheuristics*. Kluwer Academic Publishers, Dordrecht, The Netherlands, pp 1–35.

Helsgaun K (2000). An effective implementation of the Lin-Kernighan traveling salesman heuristic. *Eur J Opl Res* **126**: 106–130.

Hodgson MJ, Laporte G and Semet F (1998). A covering tour model for planning mobile health care facilities in Suhum District, Ghana. *J Regional Sc* **38**: 621–638.

Johnson DS and McGeoch LA (2002). Experimental analysis of heuristics for the STSP. In: Gutin G and Punnen AP (eds). *The Traveling Salesman Problem and its Variations*. Kluwer Academic Publishers, Dordrecht, The Netherlands, pp 369–443.

Labbe M and Laporte G (1986). Maximizing user convenience and postal service efficiency in post box location. *Belgian J Opns Res, Stat Comput Sci* **26**(2): 21–35.

Labbé M, Laporte G, Martín IR and González JJS (2004). The ring star problem: polyhedral analysis and exact algorithm. *Networks* **43**: 177–189.

Labbé M, Laporte G, Martín IR and González JJS (2005). Locating median cycles in networks. *Eur J Opl Res* **160**: 457–470.

Laporte G (1988). Location-routing problems. In: Golden BL and Assad AA (eds). *Vehicle Routing: Methods and Studies*. North-Holland, Amsterdam, The Netherlands, pp 163–197.

Lawler EL, Lenstra JK, Rinnooy Kan AHG and Shmoys DB (eds) (1985). *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. John Wiley and Sons: Chichester, UK.

Lin S and Kernighan BW (1973). An effective heuristic algorithm for the traveling-salesman problem. *Opns Res* **21**: 498–516.

Mladenović N and Hansen P (1997). Variable neighborhood search. *Comput Opns Res* **24**: 1097–1100.

Peréz JAM, Moreno-Vega JM and Martín IR (2003). Variable neighborhood tabu search and its application to the median cycle problem. *Eur J Opl Res* **151**: 365–378.

Reinelt G (1991). TSPLIB—A traveling salesman problem library. *ORSA J Comput* **3**: 376–384.

Renaud J, Boctor FF and Laporte G (1996). A fast composite heuristic for the symmetric traveling salesman problem. *Informs J Comput* **8**: 134–143.

Renaud J, Boctor FF and Laporte G (2004). Efficient heuristics for median cycle problems. *J Opl Res Soc* **55**: 179–186.

Received June 2005;
accepted December 2005 after two revisions